Machine Learning and Inductive Inference - Lecture 04

Lecture overview

Lecture 04 establishes ensemble methods as a strategy to surpass single-model performance by combining multiple predictors into a "team-based" approach, illustrated through analogies like multi-angle camera tracking and music playlist generation. The lecture defines ensembles as either homogeneous (same model type) or heterogeneous (different model types), emphasizing the necessity of generating models that make independent errors to achieve significant accuracy gains. Key techniques include voting ensembles, which aggregate predictions via majority rule under assumptions of statistical independence; stacking, which uses base learner outputs as features for a meta-learner; and bagging (Bootstrap Aggregating), which creates diverse models by training on bootstrap replicates where approximately 36.8% of data is excluded per replicate. These concepts evolve into Random Forests, which further decorrelate trees by randomly selecting features at each split, and Extremely Randomized Trees, which introduce additional randomness in split point selection to trade individual accuracy for greater diversity.

The lecture further explores boosting algorithms like AdaBoost, which iteratively combines weak learners by reweighting misclassified examples to correct previous errors, and Extreme Gradient Boosting (XGBoost), noted as a gold standard for tabular data that optimizes via gradient descent. Evaluation strategies include Out-of-Bag error estimation, which leverages excluded bootstrap samples to assess generalization without a separate test set. Theoretical foundations are provided through Bias-Variance decomposition, distinguishing between bias (underfitting due to model limitations), variance (overfitting due to sensitivity to data fluctuations), and irreducible noise, while highlighting the trade-off between model expressiveness and stability. Practical applications demonstrate these methods in multiclass classification via One-vs-Rest and Error-Correcting Output Codes (ECOC), as well as in high-stakes scenarios like the Netflix Prize, where massive stacked ensembles achieved superior recommendation accuracy by integrating diverse signal categories such as collaborative, content, and contextual data.

Slide 1: Lecture 4: Ensembles

Lecture 4 introduces the topic of ensembles within the broader context of machine learning and inductive inference. This session, presented by Sieben Bocklandt, utilizes slide decks originally developed by Jesse Davis for the Principles of Machine Learning course to explore these concepts.

Slide 2: - Deep understanding of how ensemble algorithms work

This lecture aims to provide a deep understanding of ensemble algorithms by contrasting team-based models with single-player approaches. Key topics include voting, stacking, bagging, random forests, boosting (specifically AdaBoost and XGBoost), and error-correcting output codes. Beyond mastering these techniques, students must grasp their respective strengths and weaknesses, understand the intuitive reasons why ensembles succeed, and learn how to apply them for tasks like multi-class classification using binary classifiers.

Slide 3: - Voting, stacking, bagging and random forests

This section outlines key ensemble methods, including voting, stacking, bagging, random forests, AdaBoost, and XGBoost, alongside topics on manipulating inputs/outputs, the rationale for why ensembles work, and their applications. The professor illustrates the concept of ensembles using a camera analogy, where installing multiple cameras to track movement from different angles provides a comprehensive view, similar to how combining models improves performance. This serves as a general introduction to ensemble techniques.

Slide 4: - Music playlist consisting of songs used as background music

This slide introduces a running example for the lecture on machine learning and inductive inference, focusing on a music playlist composed of songs used as background music. This specific scenario will serve as a recurring case study to illustrate core concepts throughout the course. No additional details or clarifications regarding this example are provided in the aligned transcript.

Slide 5: - Music playlist consisting of songs used as background music

This slide introduces a running example for the lecture: a music playlist containing songs used as background music. The specific objective is to develop a system capable of automatically adding new songs to this playlist, serving as a practical context for exploring machine learning and inductive inference concepts.

Slide 6: - Music playlist consisting of songs used as background music

This slide introduces a running example for the lecture: a music playlist containing songs used as background music. The specific objective is to develop a system that can automatically add new songs to this playlist, serving as a practical context for exploring machine learning and inductive inference concepts throughout the course.

Slide 7: - Music playlist consisting of songs used as background music

This running example involves a background music playlist containing positive (included) and negative (excluded) songs, with the goal of automatically adding new tracks based on features like energy and beats per minute. While individual models may disagree on whether a new song fits the playlist, this lecture addresses the challenge of multiple valid models by combining their predictions rather than selecting just one. This approach treats the collection as a "team" or "band" of models to integrate diverse classifications for more robust decision-making.

Slide 8: Faculteit, departement, dienst …

Slide 8 introduces a running example contrasting the use of one unified model against multiple separate models to represent organizational structures such as faculties, departments, and services. This comparison serves as a foundational scenario for evaluating different modeling strategies within the course context. No additional clarifications or specific examples were provided in the aligned transcript for this section.

Slide 9: Faculteit, departement, dienst …

Slide 9 introduces a running example involving organizational units such as faculties, departments, and services to explore the modeling decision between using one unified model versus multiple distinct models. This section specifically highlights "M1" as the initial reference point for comparing these alternative modeling strategies within the context of the course's ongoing case study.

Slide 10: Faculteit, departement, dienst …

Slide 10 introduces a running example contrasting the use of a single model against multiple models (M1 and M2) within an organizational context involving faculties, departments, and services. This section sets up a comparison to explore how different modeling approaches apply to these structural entities, though no specific rationale or detailed explanation is provided in the available sources for this particular slide.

Slide 11: Faculteit, departement, dienst …

Slide 11 introduces a running example contrasting the use of one unified model against multiple distinct models (labeled M1, M2, and M3) within an organizational context involving faculties, departments, and services. This section sets up a comparison to explore how different modeling approaches handle these structural entities, though no specific rationale or detailed explanation is provided in the available sources for this particular slide.

Slide 12: - An ensemble model is a predictive model that is a combination of several different predictive

An ensemble model combines several different predictive models to form a single, more robust predictor. These are classified as homogeneous ensembles when they use the same type of models, or heterogeneous ensembles when they integrate different types. This approach often yields higher accuracy than any individual constituent model, operating on the principle that "two know more than one."

Slide 13: - An ensemble model is a predictive model that is a combination of several different

An ensemble model combines several predictive models to achieve greater accuracy than any single constituent, operating on the principle that "two know more than one." These ensembles are classified as homogeneous when using the same model type in multiple ways, or heterogeneous when combining different model types. While this approach improves predictions by adding redundancy and correcting errors through diverse outputs, it raises critical questions regarding how to generate models with distinct mistakes, which specific algorithms to consider, and how to effectively combine their individual predictions into a final result.

Slide 14: Use multiple models

The lecture emphasizes the strategic use of multiple models across different organizational levels, specifically faculties, departments, and services. This approach suggests that a single model is insufficient for addressing the diverse needs within these distinct units, necessitating a tailored application of various frameworks to ensure comprehensive coverage and effectiveness throughout the institution.

Slide 15: - Assume we have k classifiers M1, M2, …, Mk

Majority voting aggregates predictions from k classifiers ($M_1, \dots, M_k$) by selecting the class predicted most frequently for a given instance x. While conceptually trivial, this method is effective; its theoretical analysis assumes all classifiers share accuracy p and make independent errors. Although the independence assumption is not always perfectly valid in practice, it serves as a foundational premise for understanding why ensemble voting improves performance.

Slide 16: Faculteit, departement, dienst …

Slide 16 introduces a voting example concerning music preferences involving three options labeled M1, M2, and M3. This scenario is presented within the context of organizational structures such as faculties, departments, or services, though no specific teacher clarifications, rationales, or detailed explanations accompany this slide in the provided sources.

Slide 17: - The question is difficult to answer in general, but we can analyze what

While the general question is difficult, analysis under simplifying assumptions shows that combining independent classifiers with equal accuracy p significantly boosts performance. Assuming mistakes are statistically independent-meaning one model's error does not influence another's-adding more models increases overall accuracy; for instance, individual accuracy of 0.8 rises to 0.896 with three classifiers, 0.988 with eleven, and 0.999 with twenty-one.

Slide 18: Faculteit, departement, dienst …

Slide 18 introduces the organizational hierarchy of "Faculteit, departement, dienst" (Faculty, Department, Service) in the context of voting procedures. As no specific definitions, examples, or clarifications were provided in the aligned transcript for this section, the note reflects only the structural terms and the voting topic explicitly presented on the slide.

Slide 19: Faculteit, departement, dienst …

Slide 19 demonstrates how ensemble voting improves classification accuracy for categories like "Faculteit," "departement," and "dienst." The data shows a clear performance increase as the number of classifiers grows: a single classifier achieves 0.800 accuracy, which rises to 0.896 with three classifiers, 0.942 with five, 0.988 with eleven, and reaches 0.999 with twenty-one classifiers. This progression illustrates that aggregating predictions from multiple models significantly enhances reliability compared to relying on a single classifier.

Slide 20: - We can prove that voting helps if we have equally good models that make

Voting ensembles are provably effective when combining equally good models that make independent mistakes, though adding more models generally works under these heavy assumptions. However, if errors are correlated or model accuracies differ, no theoretical proof exists, necessitating experimental validation to confirm performance. While the core concept of voting remains valid, this limitation motivates alternative approaches like stacking, where base learner predictions serve as input for a meta-learner to generate final outputs.

Slide 21: - Use the votes as the input of a new learner

In stacking, the predictions or votes generated by multiple base learners serve as the input features for a new meta-learner. This process effectively functions as a data transformation, where the original dataset is converted into a new representation composed of these learner outputs before being fed to the final model for prediction.

Slide 22: Faculteit, departement, dienst …

The music example illustrates stacking: predictions from several first-level models become inputs to a second-level model that learns how to combine them. The second-stage model is trained on predictions that represent the first-stage outputs, so it can learn which sources help on this task.

Slide 23: Faculteit, departement, dienst …

Slide 23 introduces a stacking example involving the terms "Faculteit," "departement," and "dienst" within the context of music. The slide text presents these organizational concepts alongside musical notation symbols ("? ? + +"), suggesting a structural analogy or specific application of stacking principles to this domain. No further clarifications, definitions, or examples are provided in the aligned transcript or source material to elaborate on this example.

Slide 24: Faculteit, departement, dienst …

Slide 24 introduces a stacking example involving the terms "Faculteit," "departement," and "dienst" within a music context. The slide visually represents this concept using question marks alongside positive and negative symbols, though no specific definitions or detailed explanations are provided in the available source material to clarify the exact nature of this stacking mechanism.

Slide 25: Faculteit, departement, dienst …

Slide 25 introduces a stacking ensemble example applied to a music dataset, illustrating how predictions from diverse base models-Decision Tree (DT), Linear model, and k-Nearest Neighbors (kNN)-are combined. The slide displays the individual outputs of these three classifiers alongside their aggregated results, demonstrating the mechanism by which stacking integrates multiple algorithmic perspectives to generate a final prediction.

Slide 26: Use the same algorithm,

To improve model performance or adapt to new contexts, maintain the identical underlying algorithm while varying the training data used. This approach isolates the impact of data composition on outcomes without introducing changes from the model architecture itself. By keeping the algorithm constant, any observed differences in results can be directly attributed to the specific variations in the dataset provided during training.

Slide 27: - Goal: multiple models that make different mistakes

To generate multiple models that make different mistakes from a single training set, we use bootstrap replicates by sampling $|S|$ examples with replacement to create new datasets (S'). This process allows specific instances to appear multiple times in a replicate while others are omitted entirely, as illustrated by the example where one item is selected twice and another is excluded. Consequently, each resulting model trains on a slightly different data distribution, enabling the ensemble to capture diverse error patterns despite originating from one source dataset.

Slide 28: - Claim: 63.2% chance example ☐is in a bootstrap replicate ☐’ with size |S|=n

There is a 63.2% probability that any specific example x appears in a bootstrap replicate S' of size n, leaving a 36.8% chance it is excluded. This result derives from calculating the probability that x is not selected for any of the n positions, which is $(1 - 1/n)^n$, and subtracting this from one. As n increases, this value asymptotically approaches $1 - 1/e \approx 0.632$. This inherent sampling variation ensures different training sets across replicates, enabling the creation of diverse models for ensemble combination.

Slide 29: - Given: Data set S, integer T

Bagging (Bootstrap Aggregating) constructs an ensemble by generating T bootstrap replicates from a dataset S, training a separate learner h_t on each replicate, and combining their predictions via an unweighted vote. For instance, if three models predict true, false, and false respectively, the final output is determined by majority vote as false. This process leverages sampling to create diverse models that are aggregated to improve overall prediction stability.

Slide 30: Faculteit, departement, dienst …

Slide 30 introduces a bagging example applied to music data. The slide title lists organizational terms such as faculty, department, and service, which appear alongside the technical topic of ensemble learning. No further explanation, rationale, or specific details regarding the music dataset or the bagging mechanism are provided in the available sources for this section.

Slide 31: Faculteit, departement, dienst …

Slide 31 introduces a bagging example applied to music data. The slide title lists organizational terms such as faculty, department, and service, which appear alongside the technical topic but are not further defined or explained in the provided source material. No additional context, rationale, or specific details regarding the music example are available in the aligned transcript or retrieval context for this section.

Slide 32: Faculteit, departement, dienst …

Slide 32 introduces a specific bagging example applied to the "music M1" dataset. The slide title lists organizational terms such as faculty, department, and service, which appear alongside this technical illustration. No further explanation, rationale, or detailed breakdown of the bagging process or the music dataset is provided in the available sources for this section.

Slide 33: Faculteit, departement, dienst …

Slide 33 introduces a bagging example applied to music data, specifically referencing models M1 and M2. The slide title also lists organizational terms such as "Faculteit," "departement," and "dienst," though the provided sources do not offer further definitions or context for these terms within this specific section.

Slide 34: Faculteit, departement, dienst …

Slide 34 introduces a bagging example applied to music data, labeled with identifiers M1, M2, and M3. The slide title also lists organizational terms "Faculteit, departement, dienst," though no further explanation or connection between these terms and the bagging example is provided in the available sources.

Slide 35: Faculteit, departement, dienst …

Slide 35 introduces a bagging example applied to music data, illustrated by three models labeled M1, M2, and M3. The slide title references organizational terms such as faculty, department, and service, though the visual content focuses exclusively on the ensemble learning scenario without further elaboration or definitions in the provided sources.

Slide 36: - Bagging works with any learner in principle

While bagging can theoretically be applied to any learner, it is most effective for unstable algorithms where minor training data variations cause significant output changes. Examples of unstable learners include decision trees, neural networks, rule learning algorithms, and 1-nearest neighbor (1NN), whereas linear regression, kNN with $k>1$, and linear threshold algorithms are stable. Bagging benefits unstable learners by using sampling to exclude noisy data points, thereby simplifying the classification task and preventing the model from overfitting to specific outliers.

Slide 37: - Suppose your data looks like this:

Bagging improves model performance by using bootstrap replicates to generate training data that excludes noisy examples, which often forces learners to expend less effort fitting outliers. This technique is most effective with unstable learners because small perturbations in the input data-caused by including or excluding specific examples during sampling-result in significant changes to the model output. Aggregating these diverse models leverages these variations to create a more robust final prediction.

Slide 38: - A variant of Bagging

Random Forests are a variant of Bagging designed to further decorrelate models by introducing feature randomness during tree construction. For each of T bootstrap replicates of the dataset, a decision tree is built without pruning; at every recursive split, the algorithm selects n features at random and chooses the best among them as the root for that subtree. This specific constraint on feature selection reduces model correlation compared to standard Bagging while maintaining the ensemble's robustness.

Slide 39: Faculteit, departement, dienst …

Slide 39 introduces a Random Forest classification example applied to music data, specifically identifying the genre "Jazz." The slide title lists organizational terms such as faculty, department, and service, which appear alongside this technical illustration. No further explanation, rationale, or additional context regarding the algorithm's mechanics or the specific dataset is provided in the available sources for this section.

Slide 40: Faculteit, departement, dienst …

Slide 40 presents a Random Forest classification example for identifying the music genre "Jazz." The model utilizes specific audio features, such as energy and beats per minute (BPM), to make its determination. A key decision rule highlighted in this example is that a BPM value less than 125 contributes to classifying the track as Jazz.

Slide 41: Faculteit, departement, dienst …

Slide 41 presents a Random Forest classification example for identifying the music genre "Jazz" using specific audio features. The model evaluates decision paths based on Energy and Beats Per Minute (BPM), where the second split specifically distinguishes genres by checking if BPM is less than 125, followed by a further condition of BPM being less than 95.

Slide 42: Faculteit, departement, dienst …

Slide 42 presents a Random Forest classification example applied to identifying the music genre "Jazz." The model utilizes specific decision rules based on features such as Beats Per Minute (BPM) and Energy, including thresholds like BPM < 125 and BPM < 95. The slide outlines three distinct feature pairings-Energy/BPM, BPM/Genre, and Energy/Genre-that contribute to the final classification output of "Jazz."

Slide 43: Faculteit, departement, dienst …

Slide 43 presents a Random Forest classification example for identifying the music genre "Jazz" using features such as Energy and Beats Per Minute (BPM). The decision logic involves evaluating relationships between these variables, specifically applying thresholds where BPM is less than 125 and further refined to BPM less than 95 to confirm the Jazz classification. This illustrates how the algorithm combines multiple feature comparisons to reach a final genre prediction.

Slide 44: Faculteit, departement, dienst …

Slide 44 presents a Random Forest classification example for identifying the music genre "Jazz" using features such as Energy and Beats Per Minute (BPM). The decision process involves evaluating specific thresholds, notably checking if BPM is less than 125 and subsequently if it is less than 95, to determine the final genre classification. This illustrates how Random Forests utilize multiple feature combinations and conditional splits to reach a prediction.

Slide 45: - Increasing n increases

Increasing the number of trees (n) creates a trade-off: it raises correlation among trees due to shared features (negative), but simultaneously improves individual tree accuracy by increasing the probability of generating accurate replicates (positive). Because these opposing effects may stabilize at a specific point, a tuning or validation set is required to identify the optimal n. Overall, random forests are computationally fast, handle large feature sets effectively through sampling, substantially reduce overfitting, and remain a gold standard that performs exceptionally well in practice.

Slide 46: - Given Dataset S, integer T

Random Forests extend bagging by building T decision trees on bootstrap replicates of dataset S, introducing randomness at each split by selecting n features without replacement and choosing the best among them, which may exclude the globally optimal split. Trees are grown fully without pruning to maximize variance reduction through ensemble aggregation. An alternative approach, Extremely Randomized Trees (Extra Trees), further increases randomness by selecting random split points for the chosen features rather than optimizing the split value, trading individual model accuracy for greater diversity within the forest.

Slide 47: Faculteit, departement, dienst …

Slide 47 presents a Random Forest classification example for identifying the music genre "Jazz." The decision rule illustrated requires two simultaneous conditions: the audio energy must be greater than 3, and the beats per minute (BPM) must be less than 130. This specific combination of thresholds serves as a single decision path within the ensemble model to categorize the input data.

Slide 48: Faculteit, departement, dienst …

Slide 48 presents a Random Forest classification example for identifying the music genre "Jazz." The decision rule illustrated requires that a track's energy level exceeds 3 while its beats per minute (BPM) remain below 130. This specific combination of thresholds serves as the criteria for assigning the Jazz label within this model instance.

Slide 49: Faculteit, departement, dienst …

Slide 49 presents a Random Forest classification example for identifying the music genre "Jazz" using audio features. The decision process involves splitting data based on Energy and Beats Per Minute (BPM), specifically evaluating conditions where BPM is less than 130 or less than 90 to determine the final genre classification.

Slide 50: Faculteit, departement, dienst …

This slide illustrates a Random Forest example classifying music as Jazz using three decision trees with splits on Energy, BPM, and Genre. For a specific track (100 BPM, 2.5 energy, Jazz), the individual trees yield mixed predictions (positive, negative, positive), which are resolved via an unweighted vote to classify it as positive. The professor notes that increasing the number of trees in the forest introduces higher correlation due to shared features across trees, which is a detrimental effect on model performance.

Slide 51: Faculteit, departement, dienst …

Slide 51 presents a Random Forest classification example for identifying the music genre "Jazz" using features such as Energy and Beats Per Minute (BPM). The model evaluates relationships between these variables, specifically applying decision rules where Jazz is identified when BPM is less than 130 or further refined to less than 90. This illustrates how the algorithm splits data based on feature thresholds to determine the target class.

Slide 52: - How well does each learned model generalize?

To evaluate how well each learned model generalizes, an alternative method called out-of-bag error is used. For every example in the database, predictions are gathered only from models that did not include that specific example during training. The final error metric is calculated by taking the majority vote of these independent predictions, comparing them to the true labels, and averaging the resulting errors across all examples.

Slide 53: - How well does each learned model generalize?

To evaluate how well learned models generalize, the out-of-bag error method assesses performance using data points excluded from training. For each database example, identify all models that did not train on it-occurring with approximately 37% probability in bootstrap replicates-and aggregate their predictions via majority vote. The final error metric is calculated by comparing these votes against true labels and averaging the resulting errors across all examples.

Slide 54: - How well does each learned model generalize?

To evaluate how well each learned model generalizes using out-of-bag error, the process examines every example in the database by identifying all models that were not trained on it. These specific models provide predictions for the example via a majority vote, which is then compared against the true label to determine the error. The final evaluation metric is calculated by taking the average of these individual errors across the dataset.

Slide 55: - How well does each learned model generalize?

To evaluate how well learned models generalize, calculate the Out-of-Bag error by testing each database example against only those models that were not trained on it. For every example, aggregate the predictions of these excluded models via a majority vote and compare the result to the true label. The final generalization metric is the average of these errors across all examples, providing an alternative evaluation method that avoids using training data for validation.

Slide 56: - How well does each learned model generalize?

To evaluate how well each learned model generalizes, the out-of-bag error method assesses performance on examples excluded from a specific model's training set. For every database example, predictions are aggregated via majority vote from all models that did not train on it, and these collective predictions are compared against the true labels. The final evaluation metric is the average of these errors across all examples, providing an alternative assessment of model generalization without requiring separate validation data.

Slide 57: - How well does each learned model generalize?

To evaluate how well each learned model generalizes, the out-of-bag error method assesses performance without requiring a separate test set. For every example in the database, predictions are aggregated from all models that did not train on that specific instance using a majority vote. The final evaluation metric is the average of the errors resulting from comparing these aggregated predictions against the true labels.

Slide 58: - How well does each learned model generalize?

Out-of-bag error evaluates model generalization by testing each instance only on models that did not train on it, using a majority vote of their predictions against the true label. For example, if three models predict an unseen yellow instance as negative while its actual label is positive, this counts as an error; averaging these errors across all instances yields the out-of-bag metric. Minimizing this error indicates strong generalization because the models correctly predict data they have never encountered, with results converging through repeated resampling in methods like random forests.

Slide 59: Learn from previous

The core principle of this section is to learn from previous mistakes within the context of faculties, departments, and services. This approach emphasizes using past errors as a foundation for improvement across these organizational units. No additional clarifications, examples, or specific rationales were provided in the aligned transcript to further elaborate on this concept.

Slide 60: - Typically, we want “strong learners”, i.e., high chance to get small error rate

While strong learners offer high accuracy with minimal error, they are difficult to train; therefore, boosting iteratively combines weak learners-models only slightly better than random guessing-to form a robust additive model. Each new model is greedily added to specifically correct the mispredictions of previous ones by adjusting weights, which increases the overall margin and confidence. This process allows test accuracy to improve even after training error reaches zero, avoiding the typical overfitting seen in other complex models.

Slide 61: - Works for binary classification problems

AdaBoost addresses binary classification by iteratively learning an additive model where each new weak learner focuses on examples misclassified by previous models. The algorithm re-weights training data to increase the importance of incorrect predictions, ensuring subsequent classifiers prioritize difficult cases. Final predictions result from a weighted vote where more accurate models receive higher coefficients (α), allowing the ensemble magnitude to reflect confidence levels beyond simple class labels.

Slide 62: Given: Training examples E, Integer T

The AdaBoost algorithm initializes training examples with equal weights and iteratively trains simple classifiers h_t for T rounds. In each iteration, it calculates the weighted error ϵ_t, determines the classifier's voting strength α_t, and then increases the weights of misclassified examples while decreasing those correctly classified to prioritize difficult cases for subsequent learners. After normalizing these weights to sum to one, the new classifier and its strength are added to the ensemble, ensuring that models performing well receive higher influence in the final prediction.

Slide 63: Faculteit, departement, dienst …

Slide 63 introduces an AdaBoost example specifically applied to music data. The slide title lists organizational terms such as faculty, department, and service, which appear alongside the technical topic but are not further defined or explained in the provided source material. No additional context, rationale, or specific details regarding the music example or the listed terms are available in the aligned transcript or retrieval context for this section.

Slide 64: Faculteit, departement, dienst …

Slide 64 presents the title "Faculteit, departement, dienst …" alongside a reference to an "AdaBoost example music h1." As no aligned professor transcript or additional explanatory text is available for this specific section, the slide's content remains limited to these headings without further elaboration on organizational structures or the AdaBoost algorithm application.

Slide 65: Faculteit, departement, dienst …

Slide 65 presents an AdaBoost example using a music dataset where the first weak learner's error rate is $\varepsilon_1 = 5/26$. This error yields a weight coefficient of $\alpha_1 \approx 0.72$, calculated via the formula $\frac{1}{2} \ln\left(\frac{1 - \varepsilon_1}{\varepsilon_1}\right)$. Consequently, the updated strong classifier function becomes $F(x) = 0.72 h_1(x)$, incorporating the weighted contribution of the first hypothesis h_1.

Slide 66: Faculteit, departement, dienst …

Slide 66 presents an AdaBoost example applied to music classification, visualizing the decision boundary of the first weak learner, $h_1(x)$, across BPM and Energy features. The plot indicates a threshold where the model predicts positive outcomes (marked with '+') for lower values and negative outcomes (marked with '-') for higher values, yielding a specific function value of $F(x) = 0.72$. This visualization illustrates how AdaBoost iteratively constructs a strong classifier by combining weak hypotheses based on feature space distributions.

Slide 67: Faculteit, departement, dienst …

Slide 67 presents an AdaBoost example applied to music classification, visualizing the decision boundary of the first weak learner, h_1, across BPM and Energy features. The diagram indicates a specific output value of $F(x) = 0.72$ for this stage, with positive and negative class predictions distributed along the axes. No additional clarifications or examples were provided in the aligned transcript for this section.

Slide 68: Faculteit, departement, dienst …

Slide 68 illustrates an AdaBoost example applied to music classification using BPM and Energy features, showing the updated strong classifier $F(x) = 0.72 h_1(x) + 1.87 h_2(x)$. The weight for the second weak learner, $\alpha_2 \approx 1.87$, is derived from its error rate $\varepsilon_2 = 1/42$ using the formula $\frac{1}{2} \ln\left(\frac{1-\varepsilon_2}{\varepsilon_2}\right)$. The slide visually contrasts the predictions of h_1 and h_2 across data points, highlighting how the algorithm combines these hypotheses to improve classification accuracy.

Slide 69: - Expect: Training error drops to 0, but test error increases

While it is typically expected that training error drops to zero while test error increases due to overfitting, the observed results contradict this trend. Even after 1000 rounds, the test error remains stable and continues to decrease even after the training error has reached zero. This phenomenon demonstrates that increasing the number of rounds does not necessarily degrade generalization in this specific context.

Slide 70: - Key idea: Error only measures if classification is correct, but confidence of

While standard error only measures classification correctness, prediction confidence is critical for difficult examples near decision boundaries. Confidence is quantified by the margin, calculated as the difference between weighted positive and negative votes, where large margins indicate strong agreement among weak learners rather than increased model complexity. Theoretically, maximizing these margins yields tighter bounds on generalization error, ensuring the model takes a decisive stance (e.g., high confidence in failure) rather than hovering uncertainly at the border.

Slide 71: - Continuous search method

Gradient descent is an iterative method for minimizing a differentiable objective. Starting from a current parameter value, move against the gradient: x(next) = x − η∇f(x), where η is the step size.

Slide 72: - Continuous search method

The y = 2x example visualizes a simple continuous objective and the gradient direction. The parameter update moves downhill; the step size controls how far each update travels.

Slide 73: - Continuous search method

Slide 73 introduces the continuous search method, specifically identifying Gradient Descent as the technique used. The slide visually associates this method with the linear equations $y=2x$ and $y=4$, alongside variable markers labeled "X," to illustrate the context of optimization in a continuous space. No additional clarifications or examples were provided in the aligned transcript for this specific section.

Slide 74: - Continuous search method

This build adds a second objective curve to the gradient-descent illustration. Each objective has its own slope and descent direction; the update rule follows the gradient of the objective currently being optimized.

Slide 75: - Continuous search method

Slide 75 introduces the continuous search method, specifically identifying Gradient Descent as the technique used to navigate continuous spaces. The slide visually associates this method with a set of linear equations ($y=2x$, $y=4$, $y=-x+10$, and $y=1.5x+2$) plotted against an X-axis, illustrating the mathematical context in which such optimization occurs. No additional verbal clarifications or examples were provided in the aligned transcript for this specific section.

Slide 76: - Continuous search method

The continuous search method utilizes Gradient Descent to optimize functions within a continuous domain. The slide illustrates this approach by presenting several linear equations, such as $y=2x$, $y=4$, $y=-x+10$, and $y=1.5x+2$, which serve as the mathematical context for applying the algorithm.

Slide 77: - Continuous search method

The continuous search method utilizes Gradient Descent to optimize functions across a continuous domain. The slide illustrates this approach using several linear equations, including $y=2x$, $y=4$, $y=-x+10$, and $y=1.5x+2$, alongside visual markers indicating the search process. Without an aligned transcript, the specific mechanics of how Gradient Descent navigates these functions remain defined solely by the slide's presentation of these mathematical examples.

Slide 78: - Continuous search method

Slide 78 introduces the continuous search method, specifically identifying Gradient Descent as the technique used to navigate continuous spaces defined by linear equations such as $y=2x$, $y=4$, $y=-x+10$, and $y=1.5x+2$. The slide visually represents this optimization process using coordinate markers (X) to illustrate how the method operates across these varying functions.

Slide 79: - Continuous search method

Gradient descent is an iterative algorithm used in continuous search methods to optimize a model by starting with initial values and progressively improving them toward a goal. The process involves calculating the gradient to determine the direction of improvement, taking a step, and re-evaluating; if steps are too large and overshoot the target, recalculating the gradient allows the algorithm to correct course and find the optimal model fit.

Slide 80: - Extreme gradient boosting

Extreme Gradient Boosting (XGBoost) extends additive ensemble models by training each new decision tree component to take a step in the direction of the negative gradient, effectively minimizing error iteratively. Although considered a niche concept inspired by deep learning principles, XGBoost remains exceptionally performant and is currently regarded as the gold standard for tabular data learning, often outperforming large neural networks on such datasets.

Slide 81: Change the output labels

Slide 81 addresses the requirement to modify output labels, specifically citing examples such as "Faculteit," "departement," and "dienst." This instruction indicates that the system's displayed terminology for organizational units must be updated to match these specific terms. No additional rationale or procedural details were provided in the aligned lecture materials for this section.

Slide 82: - Given k>2 classes: A B C …

For multiclass problems with $k>2$ classes, the one-vs-rest strategy trains k binary classifiers where each distinguishes one positive class from all others, selecting the final prediction based on the highest confidence score. Alternatively, an ensemble approach splits classes into multiple sets to create redundancy; this evolves into error-correcting output codes, where each class is represented by a unique bit code across t models. This coding scheme allows the system to tolerate individual model errors and correctly identify the class even if some predictions are incorrect.

Slide 83: - Encode by partition output labels in 2 subsets, create log(k) models

To address multiclass problems, encode output labels by partitioning them into two subsets for each binary model, requiring approximately log(k) models where k is the number of classes. This approach systematically groups labels (e.g., A,B,C,D vs E,F,G,H) across different partitions to distinguish all categories. Additionally, creating more than the minimum log(k) models introduces redundancy, functioning as "error-correcting codes" to improve robustness against individual model errors.

Slide 84: - Ensemble method for multiclass problems

This ensemble method addresses multiclass problems by constructing a T-bit codeword for each class to enable error correction. For every binary classifier h_i, the corresponding bit in a class's codeword is set to 1 if that class is assigned to the positive label set, or 0 if it is assigned to the negative label set. The process involves iterating through T steps where labels are partitioned into two disjoint sets, and a model is trained on these new binary classifications.

Slide 85: - Given example ☐, create bit vector h(☐) of length T where the ith bit:

In Error-Correcting Output Codes, given an example x, construct a bit vector $h(x)$ of length T where the i-th bit is set to 1 if the base learner $h_i(x)$ predicts positive, and 0 if it predicts negative. The final classification is determined by identifying the class whose code has the smallest Hamming distance to this generated vector $h(x)$.

Slide 86: - Given example ☐, create bit vector h(☐) of length T where the ith bit:

In Error-Correcting Output Codes, given an example x, construct a bit vector $h(x)$ of length T where the i-th bit is set to 1 if the base learner $h_i(x)$ predicts positive, and 0 if it predicts negative. The final classification is determined by identifying the class whose code has the smallest Hamming distance to this generated vector $h(x)$.

Slide 87: Explaining why ensembles work

Combining models can help when their errors differ. If predictors are reasonably accurate and their errors are not strongly correlated, averaging or voting can reduce variance; highly similar errors provide less benefit, so ensembles do not guarantee an improvement.

Slide 88: - Given a new data point x, what is the expected prediction error?

Given a new data point x drawn independently and identically from an underlying distribution p, the goal is to compute the expected prediction error defined as $E_p[(y - h(x))^2]$. Here, y represents the true label, $h(x)$ is the model's predicted label, and the expectation E_p is taken across all possible training sets. This metric focuses on the squared error, which can be mathematically decomposed into a composition of terms, though students are not required to know the derivation.

Slide 89: - Assume that the true function generating the true labels is ☐☐ and the

Assuming the true function generating labels is $f(x)$ and the average prediction of a hypothesis $h(x)$ is $\bar{h}(x)$, the expected squared error decomposes into three distinct components. This decomposition expresses the total expected error as the sum of Bias squared ($(\bar{h}(x) - f(x))^2$), Variance ($E_p[(h(x) - \bar{h}(x))^2]$), and irreducible Noise ($E_p[(y - f(x))^2]$). These terms collectively quantify how prediction accuracy is influenced by model approximation, sensitivity to training data variations, and inherent data randomness.

Slide 90: - Assume that the true function generating the true labels is ☐☐ and the

Assuming $f(x)$ is the true function generating labels and $\bar{h}(x)$ is the average prediction of $h(x)$, the expected squared error decomposes into three components: bias squared, variance, and noise. Specifically, the total error equals the squared difference between the average prediction and the true target (bias), plus the expected squared deviation of individual predictions from their average (variance), plus the irreducible noise term. Bias is defined as the model's inability to represent the true target function.

Slide 91: Faculteit, departement, dienst …

Slide 91 introduces the concept of bias in machine learning by visually contrasting the true underlying function, denoted as $f(x)$, with the learned approximation, denoted as $h(x)$. This distinction highlights the discrepancy between the actual data-generating process and the model's prediction, which is a fundamental component of understanding model error. The slide serves as a conceptual foundation for analyzing how well a learned function can represent the true relationship within the data.

Slide 92: - Inability to represent certain decision boundaries

Bias represents the inherent inability of a model to represent the true target function, often because its decision boundaries cannot match the data's shape, such as using a straight line for curved data. This error arises from incorrect assumptions, like Naïve Bayes failing when independence does not hold, or from classifiers that are "too global," such as single linear separators or small decision trees. Consequently, high bias indicates that the model is underfitting the data.

Slide 93: - Assume that the true function generating the true labels is ☐☐ and the

Assuming $f(x)$ is the true function generating labels and $\bar{h}(x)$ is the average prediction of $h(x)$, the expected squared error decomposes into three components: bias squared, variance, and noise. Bias represents the model's inability to represent the true target, while variance captures fluctuations caused by variations in the data sample. This decomposition illustrates that total expected error arises from the sum of these distinct sources of uncertainty.

Slide 94: Faculteit, departement, dienst …

Slide 94 introduces the concept of variance in the context of machine learning, visually contrasting the true underlying function $f(x)$ with a learned function $h(x)$. The slide highlights how the discrepancy between these two functions represents the variance component of model error. No additional clarifications, examples, or definitions regarding this specific comparison are provided in the accompanying transcript for this section.

Slide 95: - Statistical sources

Statistical sources of variance arise when classifiers are "too local," causing them to overfit data by fitting small subsets or reacting sensitively to minor input changes. Key contributors include computational decisions on limited data (like fine-grained tree splits), algorithmic randomization (such as neural network weight initialization), and unstable learning algorithms where decision boundaries shift with single training examples. Consequently, high variance indicates a model that fits the training data well but deviates significantly from the true underlying function.

Slide 96: - Assume that the true function generating the true labels is ☐☐ and the

Assuming $f(x)$ is the true function generating labels and $\bar{h}(x)$ is the average prediction of $h(x)$, the expected squared error decomposes into three components: bias squared, variance, and noise. Bias represents the model's inability to represent the true target, while variance captures fluctuations arising from variations in the data sample. Finally, noise reflects the inherent inability to distinguish between objects with different labels, contributing irreducibly to the total expected error.

Slide 97: Faculteit, departement, dienst …

Slide 97 introduces the concept of a true function, denoted as $f(x)$, which is obscured by noise. The slide title "Faculteit, departement, dienst …" appears alongside this mathematical representation, though no specific definitions, examples, or clarifications regarding these terms or their relationship to the function are provided in the available sources.

Slide 98: - Inherent randomness

Noise in data arises from inherent randomness in genuinely stochastic processes like equipment failure or customer behavior. It also stems from unobserved variables where critical input features are missing, measurement errors caused by sensor disturbances or improper calibration, and ambiguous labels resulting from inconsistent expert assignments. These factors collectively mean the recorded data points are imperfect representations of reality.

Slide 99: - Typically:

The bias-variance trade-off arises because bias occurs when a hypothesis class lacks good hypotheses to represent the correct target, while variance results from a class being too large and complex to fit excessive functions. Consequently, increasing expressiveness raises variance, whereas reducing it introduces higher bias. This fundamental tension is a standard interview concept in computer science and serves as a primary rationale for employing ensemble methods to balance model complexity.

Slide 100: - Ensembles can address both bias and variance!

Ensembles address both bias and variance through distinct mechanisms in bagging and boosting. Theoretically, bagging reduces variance by varying training data via bootstrapping while keeping the model's bias constant; however, in practice, it can reduce both metrics depending on whether the classifier is high-bias or high-variance. Boosting primarily reduces bias in early iterations by adding decision surfaces to increase expressiveness, whereas later iterations empirically reduce variance, though the underlying reason remains an open question.

Slide 101: - 3 possible coins

This slide introduces a Bayesian estimation problem involving three distinct coins (C1, C2, C3), each with specific probabilities of landing heads and prior selection probabilities. After selecting a coin based on its prior probability and observing a sequence of Heads followed by Tails in two flips, the objective is to calculate the conditional probability that the third flip will result in Heads, denoted as P(H|HT).

Slide 102: - 3 possible coins

This slide presents a Bayesian estimation problem involving three coins (C1, C2, C3) with distinct probabilities of heads and prior selection probabilities. After observing a sequence of Heads then Tails, the likelihoods for each coin are calculated as 0.09 for C1, 0.25 for C2, and 0.09 for C3. The objective is to use these values to predict the probability of the third flip being heads, denoted as P(H|HT).

Slide 103: - 3 possible coins

This slide illustrates a Bayesian estimation problem involving three coins with distinct head probabilities and prior selection chances. After observing a Heads-Tails sequence, the likelihoods for each coin are calculated and combined with their priors using Bayes' rule to determine posterior probabilities. The resulting normalized values show that while Coin 1 is highly unlikely (0.035), Coins 2 and 3 remain nearly equally probable candidates (0.481 and 0.485) for predicting the outcome of a third flip.

Slide 104: - 3 possible coins

To predict the third flip after observing Heads-Tails, Bayes' rule updates the prior probabilities of three coins to posterior values: C1 (0.035), C2 (0.481), and C3 (0.485). Since the evidence strongly eliminates C1 but leaves C2 and C3 nearly equally likely, we cannot rely on a single coin's specific probability for the next flip. Instead, the prediction requires combining these posterior probabilities with their respective head probabilities to calculate the general likelihood of Heads given the observed sequence.

Slide 105: - 3 possible coins

This slide illustrates a Bayesian estimation problem where one of three coins is selected based on prior probabilities, flipped twice to observe Heads-Tails (HT), and used to predict the probability of a third Head. By applying Bayes' rule, the likelihoods of observing HT for each coin are combined with their priors to calculate normalized posterior probabilities: $P(C1|HT) \approx 0.035$, $P(C2|HT) \approx 0.481$, and $P(C3|HT) \approx 0.485$. Since the specific coin remains unknown, the final prediction for the third flip is derived by summing the weighted probabilities of Heads across all coins, yielding approximately 0.68.

Slide 106: - How can the learning algorithm select among set of (almost) equally good

To select among equally good hypotheses, the Bayes optimal classifier employs a weighted majority vote of all hypotheses, where weights correspond to their posterior probabilities. This approach is provably optimal as it minimizes the error rate, and ensemble learning serves to approximate this ideal by combining predictions to achieve better accuracy than any single trusted model.

Slide 107: Faculteit, departement, dienst …

In the representation explanation, each learner operates within a specific hypothesis class that may not contain the true target function. Despite this limitation, ensemble averaging can approximate the actual target function with arbitrarily high accuracy by combining multiple learners. This approach effectively mitigates the risk of the target function lying outside any single considered hypothesis class.

Slide 108: - Most learning algorithms search through hypothesis space to find one “good”

Most learning algorithms perform heuristic searches through huge or infinite hypothesis spaces to identify a good model, a process where learners risk becoming trapped in local minima. To mitigate this, repeating the search with random restarts allows the algorithm to escape suboptimal points and potentially reach the global optimum. This strategy parallels bagging, as both approaches leverage multiple attempts to overcome difficulties caused by noisy or ambiguous examples that might otherwise mislead a single search trajectory.

Slide 109: Applications

This page is an applications section marker. It signals a shift from ensemble methods to examples of their use; it does not name a specific result on its own.

Slide 110: - The Netflix Prize Competition

The Netflix Prize Competition challenged participants to reduce the error of the Cinematch recommendation system by 10% using a dataset of 100 million ratings from 480,000 users and 17,700 films. The winning solution was a massive ensemble model that combined diverse techniques-including neighborhood models (kNN), bagged gradient boosted decision trees, temporal models, and matrix factorization-through stacking. This approach integrated five or six levels of learners, where individual predictions were fed into higher-level predictors to achieve superior accuracy.

Slide 111: - Contextual ranking through XGBoost based on:

Contextual ranking utilizes XGBoost to synthesize five distinct signal categories for recommendation optimization. These inputs include collaborative signals reflecting similar listening patterns, content signals derived from audio or semantic track properties, and user-history signals capturing both long-term preferences and recent activity. The model further incorporates contextual signals regarding time, device, session, and current activity, alongside exploration systems designed to surface unfamiliar artists or newly released content.

Slide 112: Faculteit, departement, dienst …

Motion capture is an application example in the slide sequence. It records or estimates movement from sensor or image data; this page does not imply that every motion-capture system uses an ensemble.

Slide 113: Questions?

Slide 113 marks the conclusion of the lecture segment by inviting questions from the audience. The slide displays the heading "Questions?" alongside the institutional identifiers "Faculteit, departement, dienst" and the page number 113, signaling an opportunity for students to seek clarification on the preceding material. No additional spoken commentary or specific examples were provided during this transition.

Key points to remember

The lecture further explores boosting algorithms like AdaBoost, which iteratively combines weak learners by reweighting misclassified examples to correct previous errors, and Extreme Gradient Boosting (XGBoost), noted as a gold standard for tabular data that optimizes via gradient descent. Evaluation strategies include Out-of-Bag error estimation, which leverages excluded bootstrap samples to assess generalization without a separate test set. Theoretical foundations are provided through Bias-Variance decomposition, distinguishing between bias (underfitting due to model limitations), variance (overfitting due to sensitivity to data fluctuations), and irreducible noise, while highlighting the trade-off between model expressiveness and stability. Practical applications demonstrate these methods in multiclass classification via One-vs-Rest and Error-Correcting Output Codes (ECOC), as well as in high-stakes scenarios like the Netflix Prize, where massive stacked ensembles achieved superior recommendation accuracy by integrating diverse signal categories such as collaborative, content, and contextual data.